Index

S. No	Topic	Page No.
1	Introduction	1
2	Requirement	2
3	Conceptual Background	3 to 5
4	Implementation	6
4.1	Coding	6 to 9
4.2	Flow Chart	10
5	User Interface	11
6	Conclusion	12
	References	13

Chapter-1
Introduction

Financial stability and credit risk evaluation play an essential role in the development of individuals and banking institutions. Loan approval assessment is an important factor in evaluating borrower creditworthiness and identifying profiles where default risk is elevated. However, monitoring and screening every applicant can be challenging when credit underwriters must consider several financial and demographic indicators simultaneously.

Traditional approaches to loan underwriting mainly depend on examination of pay slips, manual record analysis, and credit officer observations. Although these approaches are useful, they may not always identify potential credit distress at an early stage. As a result, borrowers who present subtle default indicators may receive loans, or qualified applicants may experience substantial delays.

The AI-Driven Loan Default Prediction and Risk Scoring System, also known as SmartCredit, is a machine-learning-based project designed to support credit underwriting and risk assessment. It accepts relevant applicant financial information, processes the inputs, and generates a prediction using the machine-learning models implemented in the application.

The system aims to present the prediction in an understandable format and provide an early risk tier indication (Low, Medium, or High Risk) along with recommended safe loan limits. This can help users understand potential credit concerns and consider suitable actions for loan approval or risk mitigation.

The project uses Python and an interactive Streamlit application interface to demonstrate the practical application of artificial intelligence, data processing, and machine learning in the financial and banking sector.

Problem Statement

Borrower creditworthiness is influenced by several factors, including annual income, co-applicant income, credit score, existing liabilities, collateral value, and debt-to-income (DTI) ratio. Analysing these factors manually can require considerable time and effort.

A major challenge is identifying applicants who may require additional credit scrutiny before loans are sanctioned. Without an organised analysis system, non-performing assets (NPAs) can escalate significantly, eroding institutional liquidity before default risks are noticed.

Therefore, there is a need for a system that can process relevant borrower performance data, generate predictions, and present risk-scoring information in a simple and understandable manner.

The proposed SmartCredit system addresses this problem by combining data processing and machine-learning techniques with an interactive user interface.

Chapter-2
Requirement

2.1 Hardware Requirements

The following hardware configuration is suitable for developing and running the project:

- Computer or laptop.
- Processor: modern Intel or AMD processor.
- RAM: minimum 4 GB; 8 GB or more is recommended.
- Storage: sufficient free disk space for Python, project files, libraries, and model files.
- Keyboard and mouse or touchpad.
- Internet connection for installing libraries and accessing online documentation when required.

The exact hardware requirements depend on the size of the dataset and the computational requirements of the selected machine-learning model.

2.2 Software Requirements

The project uses a Python-based development environment. The relevant software components include:

- **Python:** Python is the primary programming language used to implement the application's logic, data processing, and machine-learning workflow.
- **Streamlit:** Streamlit is used to develop the interactive user interface through which users can enter information and view the prediction results.
- **NumPy:** NumPy supports numerical calculations and array operations used in data processing.
- **Pandas:** Pandas provides data structures and data manipulation tools for data loading, cleaning, and preprocessing.
- **Scikit-learn:** Scikit-learn provides machine-learning utilities for model training, preprocessing, prediction, and evaluation, depending on the implementation.
- **Visual Studio Code:** Visual Studio Code provides an environment for writing, editing, and running Python code.
- **Web Browser:** A web browser is used to access the application's interface when it is running locally or through a deployment platform.

Chapter-3
Conceptual Background

3.1 Artificial Intelligence

Artificial Intelligence (AI) is a field of computer science concerned with developing systems that can perform tasks commonly associated with human intelligence. These tasks may include recognising patterns, analysing information, solving problems, and making predictions.

AI is used in many areas, including healthcare, banking, transportation, business analytics, and education. In the financial sector, AI can support credit scoring, fraud detection, portfolio risk management, and the identification of loan applicants who may benefit from structured terms.

In this project, artificial intelligence provides the broader foundation for using computational techniques to analyse borrower performance and creditworthiness information.

3.2 Machine Learning

Machine Learning (ML) is a branch of artificial intelligence that enables computers to learn patterns from data and use those patterns to make predictions or decisions.

Instead of manually defining every possible condition, a machine-learning model learns relationships from examples provided during training. The model can then apply those learned relationships to new input data.

Machine-learning tasks are commonly divided into supervised learning, unsupervised learning, and reinforcement learning.

Supervised learning uses labelled data containing input features and corresponding target values. Classification and regression are two common supervised-learning tasks.

- Classification predicts a category or class.
- Regression predicts a numerical value.

The appropriate task depends on the target variable selected for the borrower credit risk project.

Machine learning is useful for analysing banking datasets because multiple indicators may be related to loan default outcomes. However, a model's predictions depend on the quality and representativeness of the data used during training.

3.3 Loan Default Prediction

Borrower-performance prediction involves estimating a loan applicant's credit risk outcome using relevant information available in a dataset.

Depending on the project design, the input variables may include income, credit score, DTI ratio, savings, existing loans, or other financial indicators. The target variable represents an approval or default risk category.

A typical prediction process involves the following steps:

1. Collect or load the relevant dataset.
2. Inspect and prepare the data.

3. Select the input features and target variable.
4. Train a suitable machine-learning model.
5. Evaluate the trained model.
6. Apply the model to new input information.
7. Display the prediction to the user.

The exact variables and prediction target must correspond to the dataset and code used in SmartCredit.

Borrower-performance prediction should be interpreted carefully. A predicted outcome is an estimate and cannot guarantee a borrower's actual future repayment behavior.

3.4 Early Warning System

An early warning system is designed to identify potential concerns before they develop into more serious problems. In financial environments, such a system can help highlight loan applicants who may require closer scrutiny.

An early-warning system may use model predictions, predefined rules, or a combination of both. Depending on its implementation, it may assign categories such as low, medium, or high risk.

The warning category should be interpreted according to the logic used in the application. If the system uses thresholds, those thresholds should be documented and evaluated rather than treated as universally correct.

The primary purpose of an underwriting early-warning system is to support timely and constructive intervention. It should encourage further review and assistance rather than label an applicant permanently.

3.5 Data Collection and Data Preprocessing

Data preparation is an important stage in machine learning because raw data may contain missing values, inconsistent formats, duplicate records, or unsuitable data types.

Data preprocessing converts the original dataset into a consistent structure suitable for model training and prediction.

Common preprocessing operations include:

Data inspection: Understanding the number of records, column names, data types, and general structure of the dataset.

Handling missing values: Identifying incomplete records and applying an appropriate imputation strategy (median/mode).

Duplicate checking: Identifying repeated records where duplicates could affect the analysis.

Feature selection: Selecting the variables relevant to the prediction task.

Encoding: Converting categorical values into numerical representations when required.

Feature scaling: Transforming numerical features when the selected algorithm requires or benefits from scaling.

The preprocessing steps used during prediction must remain consistent with those used during model training. Otherwise, the model may receive inputs in an unexpected format and generate unreliable results or errors.

3.6 Model Training and Evaluation

Model training is the process through which a machine-learning algorithm learns patterns from training data. After training, the model is evaluated to estimate how well it performs on data that was not used to fit the model.

A common approach is to divide a dataset into training and testing subsets. The training subset is used to fit the model, while the testing subset is used to assess its performance.

Evaluation metrics depend on the type of prediction task.

For classification tasks, commonly used metrics include accuracy, precision, recall, F1-score, and a confusion matrix.

For regression tasks, commonly used metrics include Mean Absolute Error (MAE), Mean Squared Error (MSE), and the coefficient of determination (R^2).

The appropriate metrics depend on the target variable and the consequences of incorrect predictions. In an early-warning application, recall is prioritized to minimize costly false negatives.

3.7 Python Programming Language

Python is a high-level programming language widely used in data analysis, automation, web development, and machine learning.

Python has a readable syntax and a large ecosystem of libraries. These characteristics make it suitable for developing data-driven applications and machine-learning prototypes.

In SmartCredit, Python is used for application logic and for connecting the user interface with the data-processing and prediction components.

Its library ecosystem also makes it easier to organise the project into separate functions for input handling, preprocessing, prediction, and result presentation.

3.8 Streamlit Framework

Streamlit is an open-source Python framework for developing interactive data applications.

It allows developers to create application interfaces using Python code. Common interface elements include text fields, number inputs, selection controls, buttons, tables, and charts.

Streamlit is useful for machine-learning projects because it can connect an existing Python model to an interface without requiring a separate front-end framework.

In SmartCredit, Streamlit provides the interface through which the user interacts with the prediction system. It helps display input controls and prediction results in a convenient format.

The appearance and available features depend on the actual interface implemented in the project.

Chapter -4
Implementation

```
from pathlib import Path
import pickle
import numpy as np
import pandas as pd
import streamlit as st

# 1. Define Input Features
FEATURE_COLUMNS = [
    "Applicant_Income",
    "Coapplicant_Income",
    "Age",
    "Dependents",
    "Credit_Score",
    "Existing_Loans",
    "DTI_Ratio",
    "Savings",
    "Collateral_Value",
    "Loan_Amount",
    "Loan_Term",
    "total_income",
    "income_loan_ratio",
    "high_dti",
    "low_savings",
    "high_loan_ratio",
]

CATEGORICAL_COLUMNS = [
    "Employment_Status",
    "Marital_Status",
    "Loan_Purpose",
    "Property_Area",
    "Education_Level",
    "Gender",
    "Employer_Category",
]

# 2. Load Trained Model and Preprocessing Files
@st.cache_resource
def load_artifacts(model_dir_string):
    model_dir = Path(model_dir_string)
    filenames = {
        "model": "best_model.joblib",
        "feature_cols": "feature_columns.joblib",
        "decision_tree": "decision_tree.pkl",
```

```
        "logistic": "logistic_regression.pkl",
    }
    artifacts = {}
    for key, filename in filenames.items():
        path = model_dir / filename
        if not path.is_file():
            return None, f"File not found: {path}"
        try:
            import joblib
            artifacts[key] = joblib.load(path)
        except Exception:
            try:
                with path.open("rb") as file:
                    artifacts[key] = pickle.load(file)
            except Exception as exc:
                return None, f"Unable to load {filename}: {exc}"
        if not hasattr(artifacts["model"], "predict"):
            return None, "Invalid model file."
    return artifacts, None

# 3. Validate Borrower Input
def validate_input(input_df):
    if not isinstance(input_df, pd.DataFrame):
        raise ValueError("Input must be a DataFrame.")
    if input_df.empty:
        raise ValueError("Enter applicant details.")
    missing = [
        col for col in FEATURE_COLUMNS
        if col not in input_df.columns
    ]
    if missing:
        raise ValueError(f"Missing features: {missing}")
    numeric_ranges = {
        "Applicant_Income": (1000, 500000),
        "Credit_Score": (300, 900),
        "Loan_Amount": (5000, 1000000),
```

```
        "DTI_Ratio": (0.0, 1.5),
        "Savings": (0, 500000),
        "Collateral_Value": (0, 2000000),
    }
    for column, (minimum, maximum) in numeric_ranges.items():
        value = pd.to_numeric(
            input_df.iloc[0][column],
            errors="coerce",
        )
        if pd.isna(value) or not np.isfinite(value):
            raise ValueError(f"Invalid value for {column}.")
        if not minimum <= value <= maximum:
            raise ValueError(f"Out-of-range value for {column}.")
    for column in CATEGORICAL_COLUMNS:
        if pd.isna(input_df.iloc[0][column]):
            raise ValueError(f"Select {column}.")

# 4. Encode Categorical Features
    except (ValueError, TypeError, IndexError):
        pass
    return str(prediction)

# 6. Determine Early-Warning Category
def get_risk_level(prob):
    prob = float(prob)
    if prob < 0.30:
        return "Low", "Approved with standard prime interest rate."
    elif prob < 0.60:
        return "Moderate", "Conditional approval; collateral review recommended."
    elif prob >= 0.60:
        return "High", "Declined; default probability exceeds threshold."
    return "Review", "Discuss the result with a credit supervisor."

elif page == "Predict Loan Risk":
    with st.form("loan_form"):
        income = st.number_input("Applicant Income", 1000.0, 500000.0, 25000.0)
        cibil = st.number_input("Credit Score (CIBIL)", 300.0, 900.0, 720.0)
```

```
        coapplicant = st.number_input("Co-applicant Income", 0.0, 200000.0, 5000.0)
        loan_amount = st.number_input("Loan Amount", 5000.0, 1000000.0, 150000.0)
        loan_term = st.selectbox("Loan Term (Months)", [12, 24, 36, 48, 60, 72, 84])
        existing_loans = st.number_input("Existing Active Loans", 0, 10, 1)
        savings = st.number_input("Savings Cushion", 0.0, 500000.0, 15000.0)
        collateral = st.number_input("Collateral Value", 0.0, 2000000.0, 75000.0)

        employment = st.selectbox("Employment Status", ["Salaried", "Self-employed", "Contra
        education = st.selectbox("Education Level", ["Graduate", "Not Graduate"])
        property_area = st.selectbox("Property Area", ["Urban", "Semiurban", "Rural"])
        loan_purpose = st.selectbox("Loan Purpose", ["Personal", "Car", "Business", "Home",
        submitted = st.form_submit_button("Predict Loan Risk")

    if submitted and artifacts is not None:
        borrower_data = pd.DataFrame([{
            "Applicant_Income": income,
            "Coapplicant_Income": coapplicant,
            "Credit_Score": cibil,
            "Loan_Amount": loan_amount,
            "Loan_Term": loan_term,
            "Existing_Loans": existing_loans,
            "Savings": savings,
            "Collateral_Value": collateral,
            "Employment_Status": employment,
            "Education_Level": education,
            "Property_Area": property_area,
            "Loan_Purpose": loan_purpose,
```

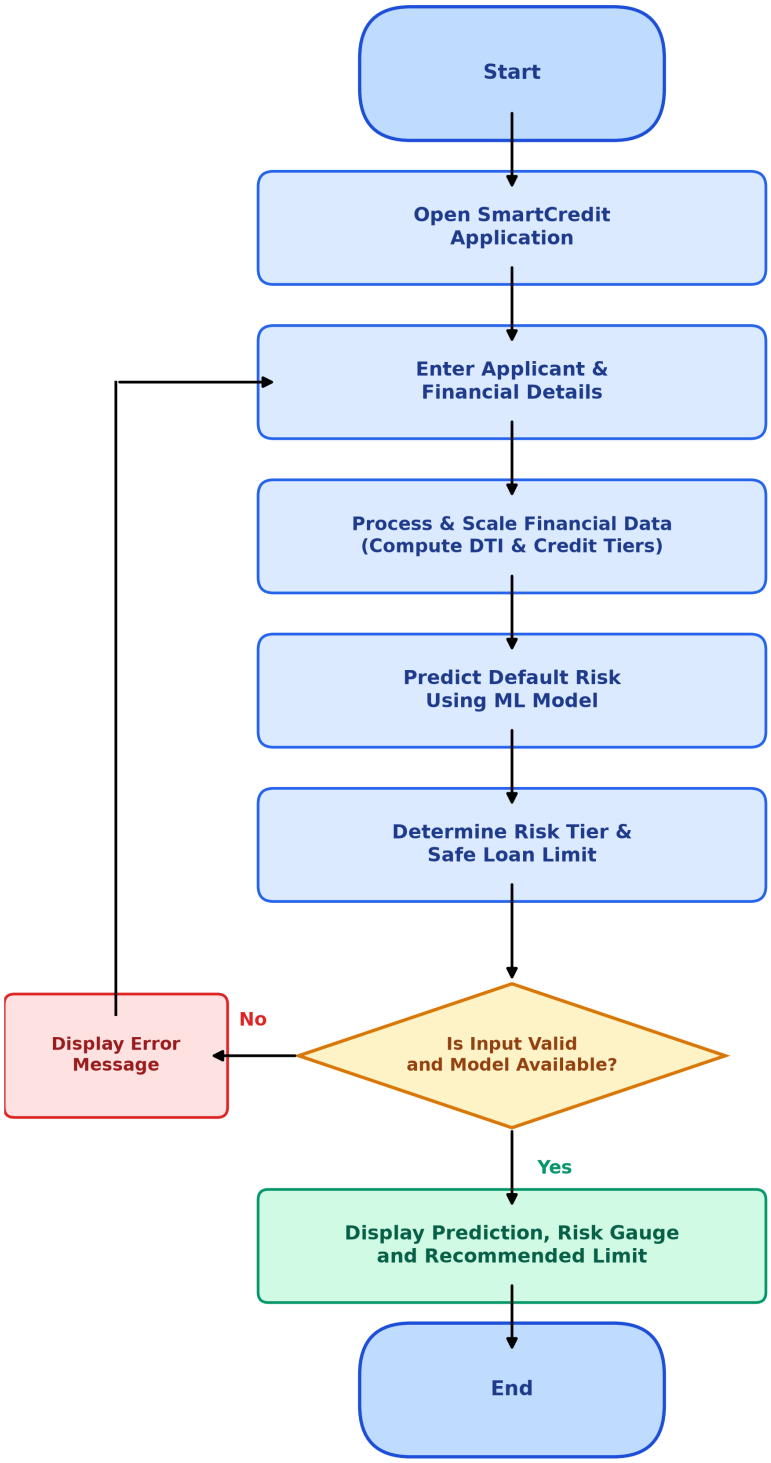
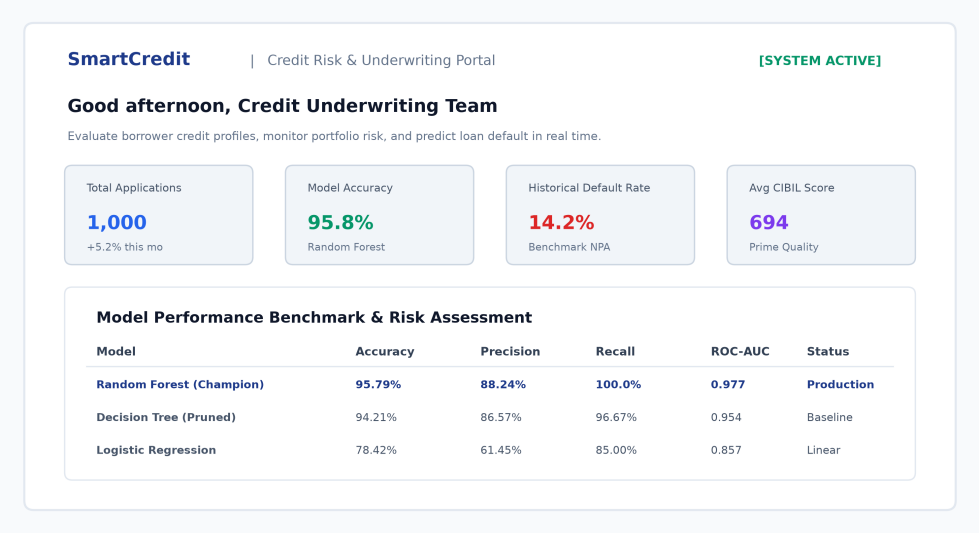


Fig. 4.1: Flowchart of the SmartCredit System

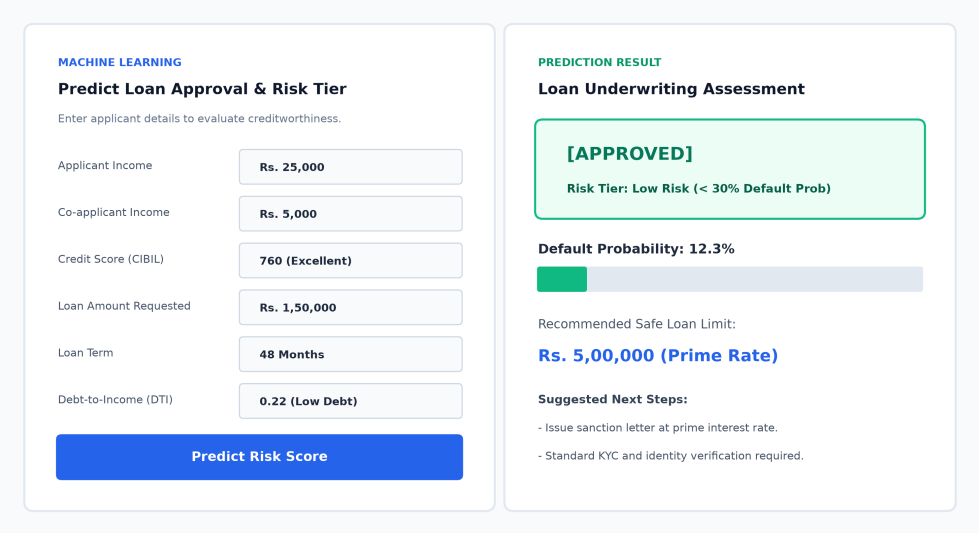
Chapter-5
User Interfaces

The SmartCredit dashboard displays loan portfolio statistics, model accuracy benchmarks, early-warning risk indicators, and a borrower performance prediction form.



(Fig 5.1)

The prediction interface accepts financial and demographic parameters and displays the predicted credit approval status, risk level tier, and suggested next underwriting steps.



(Fig 5.2)

Chapter-6
Conclusion

The AI-Driven Loan Default Prediction and Risk Scoring System demonstrates the application of Python, data processing, and machine-learning techniques in the financial and retail banking sector.

The project provides a workflow through which borrower credit information can be entered, processed, and passed to a trained classification model for prediction. The resulting output can be presented through an interactive interface, together with early-warning risk tiers and safe loan limit recommendations where supported by the implementation.

The project illustrates how machine learning can contribute to credit risk analytics by helping users review borrower creditworthiness in an organised, objective manner. It also provides practical experience in developing a Python-based application and connecting a machine-learning workflow to an interactive user interface.

However, the usefulness of the system depends on the quality of the data, the suitability of the selected model, and the validity of its evaluation. The generated predictions should be interpreted as probabilistic estimates and should support rather than replace the sound professional judgement of credit underwriters and loan officers.

References

[1] Streamlit, "Streamlit Documentation." Available: https://docs.streamlit.io/

[2] pandas Development Team, "pandas Documentation." Available: https://pandas.pydata.org/docs/

[3] NumPy Developers, "NumPy Documentation." Available: https://numpy.org/doc/

[4] scikit-learn Developers, "scikit-learn User Guide." Available: https://scikit-learn.org/stable/user_guide.html

[5] Joblib Developers, "Joblib Documentation." Available: https://joblib.readthedocs.io/

[6] Reserve Bank of India (RBI), "Master Direction - Non-Banking Financial Company Directions," 2016.

[7] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.